Snake Game Documentation

Introduction:

Suppose you are waiting on a webpage to load due to high traffic request, or waiting for an image to open, what is one thing to do to avoid brain rot and still keeps you focused -> you simply play a game

Thus comes the snake game, which requires from you fast response and focus, thus saving your energy needed to recharge your brain.

But what if you want to design your own snake game, what exactly do you need ?

How to think :

When I say snake game, what comes first to your mind ?

Snake comes at first, the food that that snake is triggered, the walls that you try to avoid of course.

Well, not bad , but you think as a player or to be much more exact, as a semi-implementer, not a full one.

Let me tell you how did I think of it , after watching some videos, it consists of states (such as the finite state machine , which to be with me on the line -> the program consists of some states and transitions between the states) , and those states are managed together in order to have a full program.

Thus to speak in the context of being a gamer, although I am not a gamer, we have screens, one screen for the main menu , another

one for the play screen , and another one …. or let us implement those only and we might think of another screens to have.

So simply to recap ,we want to have screens (which are have the 2 screens) , and we want a manager to manage between screens (maintaining the transitions between them)

Inside the game play , we need to have some animations , as the snake, the grass that the snake will move on , the food, and the walls.

Finally, in order not to have a game as the one that good opens to you when you don't have an internet, we need to have a gui that makes it somehow interesting.

So at the end, here is what we have came up with:

1. Screens
2. Screen Managers
3. Animations on the screenplay (Snake, Food,…)
4. GUI

After we think, how to start implementation? :

So, at first we want to have a container for the screens, at which this could be a class called Screen, at which we will inherit from it the 2 screens .

Then, we have the class ScreenManger that will mange our screens, and don't forget about the gui (in my project, I have used SFML libraries, but you could use anything)

For the SFML , we need to have a font to use in order to write on the window of the game, and textures to use (texture is simply an image that is mapped into the game , as for example , if you have an image of your friend that you want to troll , you could simply save it , provide it to the function addTexture (inside our program),

and using the Texture header inside the gui files, you could add it inside the game.

Then at last, the snake itself needs a class, at which we could control its movement , increase in size , and know when it is on food ,or when it will collide with a wall .

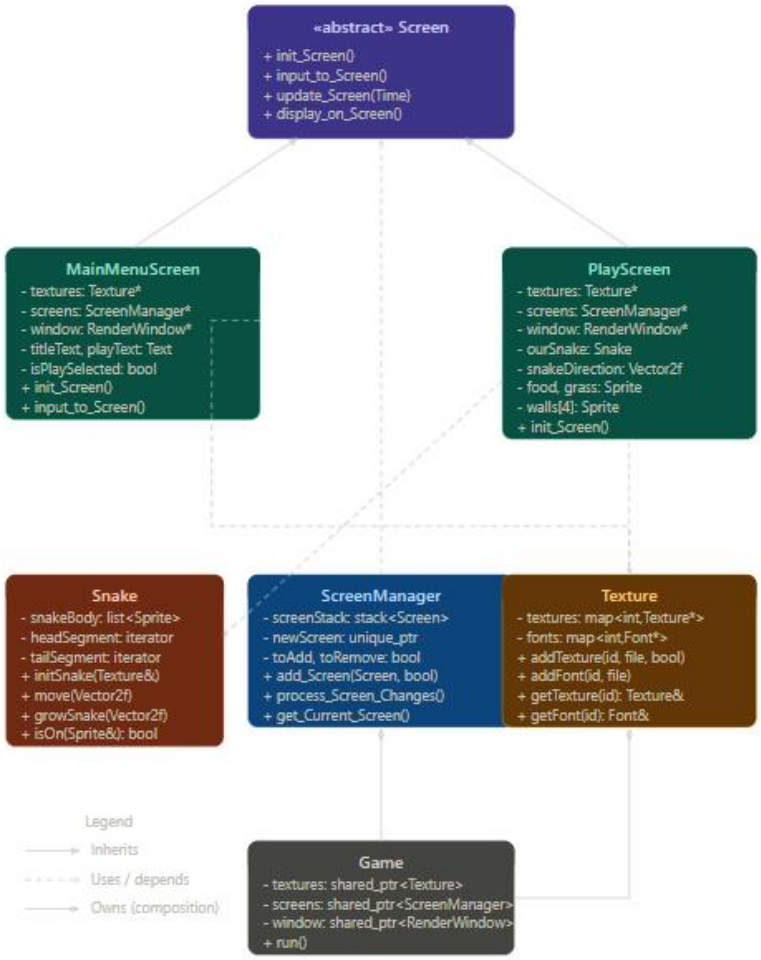
At last, we need a starting point, from which we could start our program, which is a class as well called Game.

Thus the output from this stage is that we need to have those classes :

1. Screen class
2. ScreenManager class
3. MainMenu & GamePlay screens
4. Texture class
5. Snake class

Let us implement:

Here is the class diagram of our classes that we have talked about :



What to notice from this class diagram:

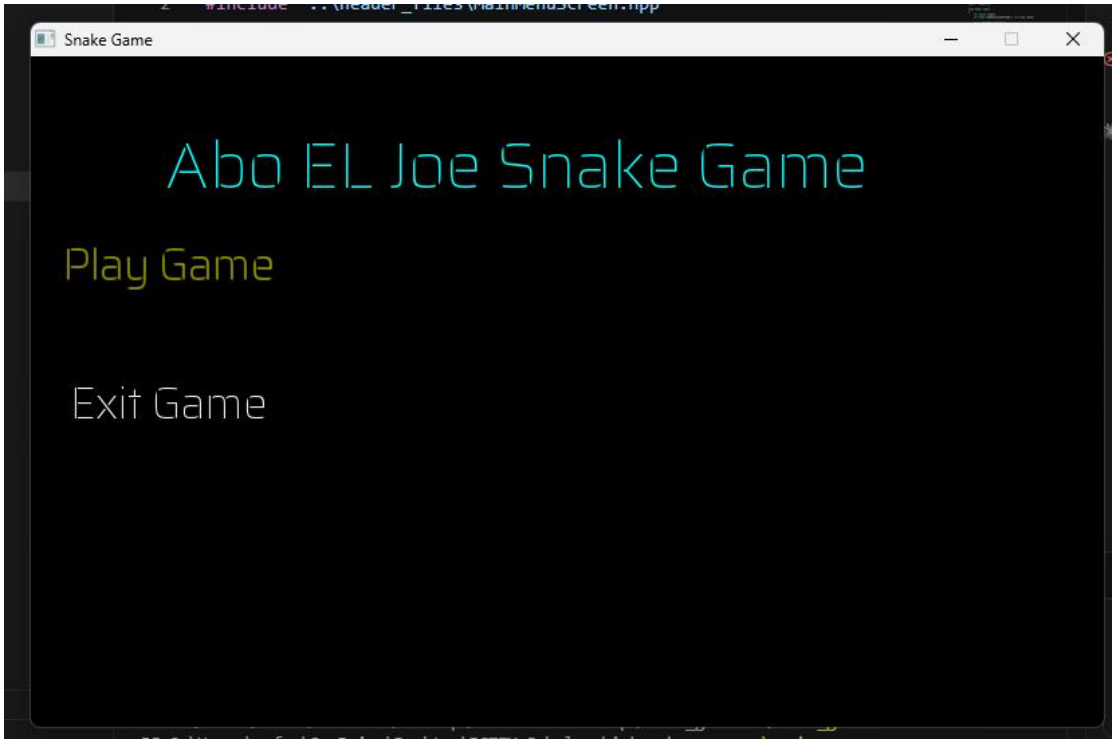
1. MainMenuScreen and PlayScreen both inherit from the abstract Screen base class, which defines the four virtual methods every screen must implement.

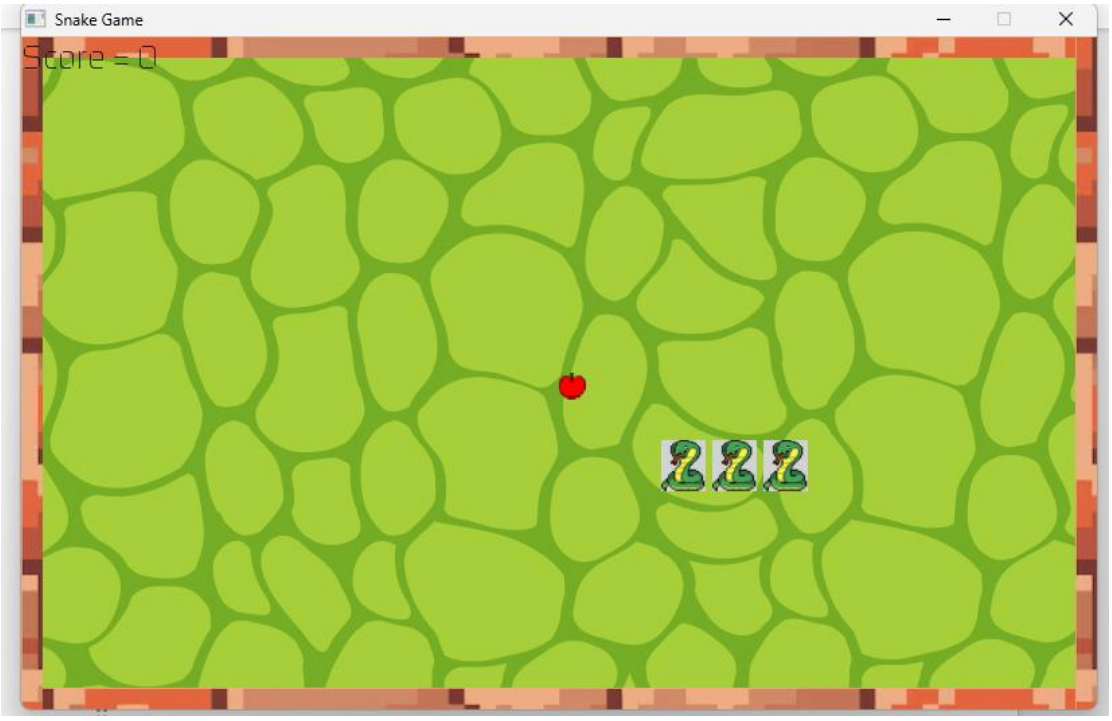
2. Composition (owns) — Game owns ScreenManager, Texture, and the RenderWindow via shared_ptr. These live and die with the Game object.

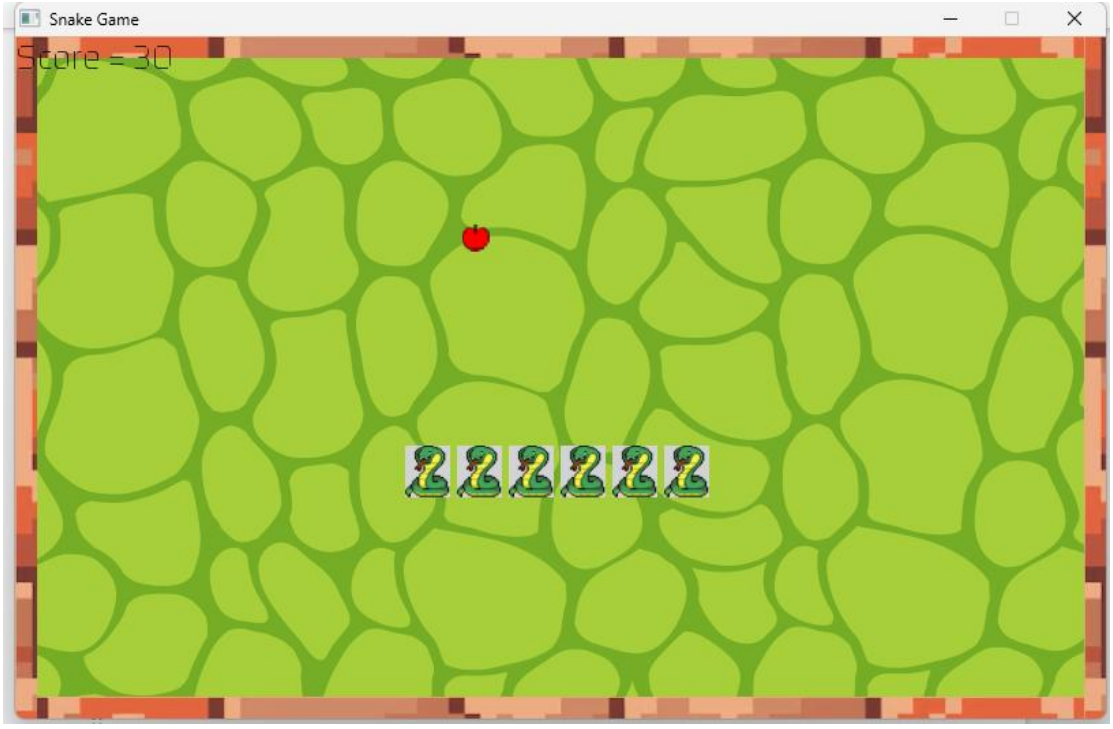
3. Uses/depends — PlayScreen depends on Snake and Texture. MainMenuScreen depends on Texture. Both screens hold raw pointers to ScreenManager and the window (they don't own them, Game does).

4. ScreenManager manages the screen stack and holds a unique_ptr<Screen> to the pending screen during transitions.

Screenshots from the game :







Game Behavior Notes :

1. Notice that there isn't 180 degree movement (meaning that if the snake is moving down , you can't press up button , and expect it to move up 180 degrees) , the snake only moves 90 degree movements.

2. Whenever you hit the wall , you will return back to the main menu as the game is over (an optimization that I might make is simply to go to game over screen and show the results outside, but this is just the first version of the game.)